Architecture et Systèmes d'Exploitation : Gestion des Processus, Cycle de Vie et Signaux POSIX

Thème principal : Gestion des processus sous Unix/Linux, cycle de vie (Élu, Prêt, Bloqué), filiation (fork/exec) et signaux POSIX

Synthèse exécutive : Ce cours aborde l'architecture interne des systèmes d'exploitation de type Unix/Linux en étudiant le concept fondamental de processus. Il explique la transition d'un programme statique stocké sur le disque dur vers un processus dynamique s'exécutant en mémoire RAM. Afin de permettre l'exécution simultanée de multiples applications sur un nombre limité de cœurs processeurs, le noyau Linux utilise le multitâche préemptif et gère le cycle de vie des processus à travers trois états fondamentaux (Élu, Prêt, Bloqué). La séance décortique le bloc de contrôle de processus (PCB), la structure généalogique parent-enfant (PID/PPID) issue du processus initial systemd/init (PID 1), le mécanisme de création par les appels système fork() et exec(), ainsi que la communication asynchrone et le contrôle des tâches au moyen des signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour exécuter une application, le système d'exploitation doit faire basculer le code d'un état statique sur le disque dur vers un état actif en mémoire vive (RAM). Ce passage crée une entité dynamique appelée processus. Dans un système informatique moderne, le nombre de programmes à exécuter dépasse largement le nombre de cœurs processeurs physiques. Le noyau Linux orchestre la simultanéité des tâches grâce au multitâche préemptif : l'ordonnanceur (scheduler) du noyau ne laisse pas un processus occuper le processeur indéfiniment. Il découpe le temps processeur en tranches ultra-courtes (quanta de temps) qu'il alloue successivement aux processus. Le cycle de vie d'un processus s'articule autour de trois états majeurs : 1. **Élu (Running)** : Le processus occupe physiquement un cœur du processeur et ses instructions machine sont exécutées. 2. **Prêt (Ready)** : Le processus possède l'intégralité de sa structure en mémoire RAM et attend que l'ordonnanceur lui attribue son prochain quantum de temps processeur. 3. **Bloqué (Waiting / Blocked)** : Le processus déclenche une requête nécessitant une attente externe (par exemple une lecture de fichier sur le disque dur, la réception d'un paquet réseau ou l'attente d'une saisie utilisateur au clavier). Le noyau libère immédiatement le processeur pour un autre processus et place la tâche en attente d'événement. Dès que l'événement survient et que la donnée devient disponible, le processus ne retourne JAMAIS directement à l'état Élu : il bascule impérativement dans l'état Prêt pour être réinscrit dans la file d'attente de l'ordonnanceur.

- • Programme vs Processus : Un programme est un fichier exécutable statique et passif stocké sur le disque dur (ex: /usr/bin/ls). Un processus est son instance dynamique active chargée en mémoire RAM, consommant du temps CPU et possédant des ressources attribuées.
- • Multitâche Préemptif & Ordonnanceur : Le noyau (kernel) découpe le temps processeur en minuscules quanta de temps et les alloue dynamiquement aux processus éligibles pour simuler une exécution simultanée.
- • Bloc de Contrôle du Processus (PCB) : Structure de données interne du noyau qui conserve l'état du processeur, les registres, la carte mémoire (stack, heap, code, data), la priorité, les identifiants (PID, PPID, UID) et les descripteurs de fichiers ouverts pour chaque processus.
- • Modèle des 3 États du Processus : Élu (Running - occupe le CPU et exécute son code), Prêt (Ready - dispose de toutes ses ressources RAM et attend qu'un quantum CPU lui soit attribué), Bloqué (Waiting/Blocked - suspendu en attente d'une opération d'Entrée/Sortie ou d'un événement externe). Un processus Bloqué repasse toujours par l'état Prêt avant de redevenir Élu.

Cas concret enseignant : *Analogie du livre de recettes de cuisine : Le programme stocké sur le disque dur est comparable à un livre de recettes sagement rangé dans une bibliothèque (passif, statique, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, occupant de la place en mémoire RAM sur le plan de travail, consommant du temps processeur CPU pour la cuisson et manipulant des ustensiles équivalant aux descripteurs de fichiers).*

2. Filiation, Arborescence et Mécanismes de Création (fork et exec)

Au sein des systèmes d'exploitation Unix/Linux, aucun processus ne peut apparaître de manière spontanée. Tous les processus utilisateurs s'inscrivent dans une généalogie stricte. Lors de l'initialisation de l'ordinateur, le noyau démarre le tout premier processus utilisateur, nommé systemd (ou init sur d'anciennes distributions), qui reçoit de façon garantie le PID 1. Lorsqu'un utilisateur lance une commande dans un terminal Bash (comme 'ls' ou 'vim'), le shell interactif (processus parent) exécute la création de la sous-tâche en deux temps : 1. **fork()** : Le processus père exécute l'appel système fork(). Le noyau duplique alors le père à l'identique. Un nouvel identifiant (PID) est assigné au fils, et le PPID du fils prend la valeur du PID du père. 2. **exec()** : Immédiatement après la bifurcation, le processus fils appelle l'une des fonctions de la famille exec(). Le noyau remplace l'intégralité du code et des données hérités de l'enfant par les nouvelles instructions binaires du programme visé. Le processus conserve son PID mais exécute désormais le nouveau code.

- Arborescence et Identifiants (PID / PPID) : Chaque processus possède un Process Identifier (PID) unique attribué par le noyau et pointe vers le Parent Process ID (PPID) du processus qui l'a créé. Tous les processus forment un arbre généalogique unique dont la racine est systemd ou init (PID 1).
- Appel système fork() : Fonction du noyau qui duplique le processus parent à l'identique pour créer un processus enfant possédant son propre PID et une copie exacte du contexte mémoire.
- Appel système exec() : Fonction du noyau qui remplace l'image mémoire du processus appelant par le code binaire d'un nouveau programme à exécuter.

3. Contrôle des Tâches et Communication Inter-Processus : Les Signaux POSIX

Les signaux POSIX représentent la forme de communication inter-processus (IPC) la plus élémentaire sous Unix. Lorsqu'un signal est adressé à un processus cible, le noyau interrompt immédiatement le fil séquentiel des instructions pour exécuter l'action requise. Face à un signal, trois comportements sont possibles pour le processus : 1. Exécuter l'action par défaut du système (souvent la terminaison de la tâche). 2. Ignorer le signal (ignore). 3. Intercepter le signal (catch) en associant une fonction personnalisée (le gestionnaire de signal). Cela permet par exemple à un traitement de texte d'intercepter SIGTERM pour afficher 'Voulez-vous enregistrer votre document ?' avant d'éteindre l'application. Toutefois, afin de préserver l'autorité absolue de l'administrateur système et permettre de neutraliser un programme défaillant ou bloqué dans une boucle infinie, la norme POSIX définit deux exceptions incapturables : **SIGKILL (Signal 9)** et **SIGSTOP (Signal 19)**. Ces deux signaux sont traités en direct par le noyau lui-même. Aucun code applicatif ne peut les détourner ou les ignorer.

- Signal POSIX : Interruption logicielle asynchrone transmise par le noyau à un processus pour lui notifier un événement ou lui ordonner une instruction.
- Gestionnaire de signal (Signal Handler) : Routine logicielle enregistrée par le développeur pour intercepter (catch) un signal et en exécuter un traitement sur-mesure au lieu du comportement système par défaut.
- SIGINT (Signal 2) : Déclenché par 'Ctrl + C' dans le terminal. Demande une interruption douce et modérée d'une tâche interactive.
- SIGTERM (Signal 15) : Signal d'arrêt propre envoyé par défaut par la commande 'kill '. Il invite le processus à libérer ses ressources, fermer ses connexions et sauvegarder ses fichiers avant de s'arrêter.
- SIGKILL (Signal 9) : Ordre d'extinction inconditionnel et immédiat exécuté directement par le noyau Linux. STRICTEMENT IMPOSSIBLE à intercepter (catch), masquer ou ignorer.
- Job Control (SIGSTOP 19 / SIGCONT 18) : 'Ctrl + Z' émet SIGSTOP pour suspendre l'exécution d'une tâche ; les commandes 'fg' (foreground) ou 'bg' (background) émettent SIGCONT pour reprendre son exécution.

Cas concret enseignant : *Analogie de l'expulsion du bâtiment : SIGTERM (Signal 15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de ranger vos affaires et d'évacuer (vous disposez du temps nécessaire pour sauvegarder vos fichiers et fermer la porte à clé). SIGKILL (Signal 9) représente l'intervention de la brigade de sécurité d'urgence qui défonce la porte et vous dégage sur-le-champ sans vous laisser la moindre seconde pour toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal Linux alors que la commande 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : La commande 'kill' attend la syntaxe suivante : 'kill - '. Lorsque vous saisissez 'kill 9 1234', le shell interprète '9' et '1234' comme deux identifiants de processus (PID) distincts auxquels il doit envoyer le signal par défaut (SIGTERM - 15). Il tente donc d'arrêter le processus PID 9 et le processus PID 1234. Le tiret '-' est indispensable pour indiquer à l'utilitaire kill que le nombre qui suit correspond au numéro de signal (Signal 9 = SIGKILL) et non à un PID.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire avec la commande kill ?
Réponse du professeur : Un processus Zombie est un processus enfant dont l'exécution est terminée (il a libéré sa mémoire RAM et ses ressources CPU), mais dont la ligne persiste dans la table des processus du noyau parce que son processus parent n'a pas encore lu son code de retour via l'appel système wait(). On ne peut PAS détruire un zombie avec 'kill -9' car le processus est déjà mort ! Pour éliminer un zombie, il faut forcer le parent à lire son statut (en lui envoyant le signal SIGCHLD), ou bien tuer le processus parent. Si le parent est tué, le zombie devient 'orphelin' et est immédiatement adopté par le processus racine systemd/init (PID 1), qui nettoie automatiquement sa ligne dans la table des processus.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un administrateur effectue la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle exact du symbole '&', détaillez ce qui se passe lors de l'exécution de la commande kill -15 et décrivez le résultat obtenu lors d'un contrôle ultérieur par la commande ps.

Démarche de résolution :
Étape 1 : Le symbole '&' placé à la fin de la commande 'sleep 600 &' indique au shell interactif d'exécuter la tâche en arrière-plan (background). Le shell affiche instantanément l'identifiant de job (ex: [1]) ainsi que le PID attribué par le noyau (ex: 4521) et rend immédiatement la main à l'utilisateur sans bloquer l'invite de commande. Étape 2 : La commande 'ps -ef | grep sleep' filtre la liste globale de tous les processus exécutés sur le système. La sortie confirme que le processus 'sleep 600' est actif avec le PID 4521 et identifie son PPID (qui correspond au PID du shell Bash hôte). Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM (Signal 15) au processus 4521. L'utilitaire sleep ne possédant pas de gestionnaire spécifique pour capturer ou ignorer SIGTERM, l'action par défaut du système d'exploitation s'applique : l'arrêt propre et la suppression du processus.

Résultat : Le processus 'sleep 600' est détruit de manière propre par le signal SIGTERM (Signal 15). Une vérification ultérieure au moyen de la commande 'ps -ef | grep sleep' confirme sa disparition complète de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement du système ?

R : Le premier processus utilisateur est 'systemd' (ou 'init' sur les anciennes distributions Linux) et son PID est impérativement 1.

Q : Quel signal POSIX est généré et transmis au processus au premier plan lorsqu'un utilisateur presse la combinaison 'Ctrl + C' dans son terminal ?

R : La combinaison 'Ctrl + C' génère le signal SIGINT (Signal 2), qui demande l'interruption modérée de la tâche.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté par un gestionnaire de signal écrit par un développeur ?

R : Parce que la norme POSIX impose que SIGKILL soit intercepté et exécuté directement par le noyau (kernel) lui-même, garantissant à l'administrateur système le pouvoir absolu de détruire n'importe quel processus défaillant sans que le programme ne puisse le bloquer.